M06: Access Control: Permissions

> "Your deploy script won't run. Your web page shows 403. Your database password is exposed. Three emergencies -- one root cause."

Quick Check

Before we dive in:

- Who has seen `Permission denied` on any computer?
- Who has ever been locked out of a file they *knew* was there?
- Who has heard of `chmod` or `sudo`?

> No wrong answers. By the end of this module, `Permission denied` will never confuse you again.

What You'll Walk Away With

By the end of this module, you'll be able to:

- **Read any permission string**

  From `ls -l` and know exactly who can do what

- **Use `chmod`**

  To fix "Permission denied" errors in seconds

- **Use `chown`**

  To fix ownership problems that break web apps

- **Use `sudo`**

  To run commands that require administrator access

- **Troubleshoot**

  The three most common permission emergencies

Who Gets Access? Three Levels

Every file on Linux has three audiences:

User (Owner)

The person who owns the file (usually whoever created it)

Group

A set of users who share access (like a team)

Others

Everyone else on the system

> Remember the deploy script that won't run? `ls -l` will show you exactly *who* is locked out and *why*.

What Can They Do? Three Permissions

Each level gets three possible permissions:

Permission	On a File	On a Directory
r (read)	View contents	List contents with `ls`
w (write)	Modify contents	Create or delete files inside
x (execute)	Run as a script	Enter with `cd`

> The one that trips everyone up: `x` **on a directory** means "allowed to `cd` into it" -- not "execute the directory."

Reading Permissions: `ls -l`

You already know `ls`. The `-l` flag shows permissions, ownership, and more:

```
ls -l config.txt
```

```
-rw-r--r--  1  anas  developers  2048  Jan 15 09:30  config.txt
```

- **First character** -- file type: `-` = file, `d` = directory, `l` = link
- **Next 3** (`rw-`) -- **Owner** permissions: read + write
- **Next 3** (`r--`) -- **Group** permissions: read only
- **Last 3** (`r--`) -- **Others** permissions: read only

> Read it like a sentence: "Owner can read and write. Group can read. Others can read. Nobody can execute."

The Number System: r=4, w=2, x=1

Each permission has a value. **Add them up** per level:

- **r** = 4, **w** = 2, **x** = 1

One example makes it click -- `chmod 754`**:**

```
7          5          4
Owner      Group      Others
4+2+1      4+0+1      4+0+0
rwx        r-x        r--
```

- Owner can do everything
- Group can read and execute (not modify)
- Others can read only

> Don't memorize all combinations. Just remember **4, 2, 1** and add. That's the whole system.

Permission Patterns You'll Actually Use

Number	Letters	When to Use
755	`rwxr-xr-x`	Scripts and directories -- owner edits, everyone else runs
644	`rw-r--r--`	Config files -- owner edits, everyone else reads
600	`rw-------`	Secrets and credentials -- owner only, everyone else locked out
700	`rwx------`	Private directories -- owner only

> Don't memorize the table. **Know these four and you'll cover almost every real situation.**

First, Meet `sudo`

`sudo` = "Super User DO" -- runs one command with administrator (root) privileges

```
sudo ls /root
```

- Linux follows **least privilege** -- you work as a normal user by default
- Some actions (changing file ownership, reading system files) require administrative access
  `sudo` gives you root power for **one command**, then drops back to normal
- The system will ask for **your password** the first time

> Think of `sudo` as a master key. You don't carry it around -- you use it only when a specific door is locked.

Changing Permissions: `chmod`

`chmod` (**ch**ange **mod**e) -- sets who can read, write, and execute

Numeric method -- set exact permissions:

```
chmod 755 deploy.sh        # Owner: all. Group/Others: read + execute
chmod 600 secrets.env      # Owner: read + write. Everyone else: nothing
```

Symbolic method -- add or remove one permission:

```
chmod +x deploy.sh         # Add execute for everyone
chmod o-r secrets.env      # Remove read from others
```

> Remember the deploy script emergency? `chmod +x deploy.sh` -- that's the fix. One command.

Changing Ownership: `chown`

`chown` (**ch**ange **own**er) -- sets which user and group own a file

- Requires `sudo` because only root can change ownership

```
sudo chown alice file.txt           # Change owner to alice
sudo chown alice:devs file.txt      # Change owner AND group
sudo chown :devs file.txt           # Change group only
sudo chown -R alice:devs /opt/app/  # Recursive -- directory and everything inside
```

> The 403 Forbidden emergency? The web server runs as `www-data` but the files were owned by someone else. Fix: `sudo chown www-data:www-data /var/www/html/index.html`

Three-Step Permission Fix

When you see `Permission denied`, check in this order:

01

Who owns it?

Run `ls -l filename`

02

What are the permissions?

Decode the permission string

03

Who am I?

Run `whoami` to see your current username

04

Decide the fix

Do you need `chmod` (change permissions) or `chown` (change ownership)?

Emergency	Root Cause	One-Line Fix
Script won't execute	No `x` permission	`chmod +x deploy.sh`
Web app shows 403	Wrong file owner	`sudo chown www-data:www-data file`
Credentials exposed	Permissions too open	`chmod 600 secrets.env`

A Note on SSH Keys

- SSH keys are used for **secure server connections** -- you'll learn about SSH in a later module
- For now, just know: SSH key files require **exactly** `600` permissions
- If permissions are too open, SSH refuses to use the key (a security feature)

```
chmod 600 ~/.ssh/id_rsa
```

> Don't worry about SSH details yet. Just remember: **private keys need** `600`. That one fact will save you a debugging session someday.

Aha! Moment 💡

Think about what happens without proper permissions:

- **Database credentials readable by everyone** → Attacker dumps your database
- **Web server can't read its own files** → 403 errors, site down
- **Log files writable by web app** → Attacker injects fake logs, covers tracks
- **Scripts with 777 (all permissions)** → Any user can modify code before execution

Now think about defense in depth:

```
chmod 600 ~/.aws/credentials     # AWS keys: owner only
chmod 644 /etc/nginx/nginx.conf  # Nginx config: owner writes, all read
chmod 755 /usr/local/bin/deploy  # Deploy script: owner modifies, all execute
chmod 700 ~/.ssh/                # SSH directory: owner only
```

Real-world security:

- **Least privilege principle:** Give minimum access needed
- **Separation of concerns:** Web server runs as `www-data`, not root
- **Defense in depth:** Even if attacker gets in, permissions limit damage

> **Every security audit starts here.** PCI-DSS, SOC 2, ISO 27001 — they all check file permissions. This isn't optional — it's how you stay employed.

Permissions Cheat Sheet

Common permission patterns:

```
chmod 755 script.sh              # Scripts: owner full, others read+execute
chmod 644 config.txt             # Configs: owner write, others read
chmod 600 secrets.env            # Secrets: owner only
chmod 700 ~/.ssh/                # SSH directory: owner only
chmod 600 ~/.ssh/id_rsa          # Private keys: owner read+write ONLY
chmod 644 ~/.ssh/id_rsa.pub      # Public keys: owner write, all read
```

Permission number breakdown:

- **7** = rwx (read + write + execute) = 4+2+1
- **6** = rw- (read + write) = 4+2
- **5** = r-x (read + execute) = 4+1
- **4** = r-- (read only) = 4
- **0** = --- (no access) = 0

Symbolic method:

```
chmod +x file                    # Add execute for everyone
chmod -w file                    # Remove write for everyone
chmod u+x file                   # Add execute for user (owner)
chmod g-w file                   # Remove write for group
chmod o-r file                   # Remove read for others
chmod a+r file                   # Add read for all (u+g+o)
```

Ownership:

```
sudo chown alice file            # Change owner
sudo chown alice:devs file       # Change owner and group
sudo chown :devs file            # Change group only
sudo chown -R alice:devs dir/    # Recursive (directory + contents)
```

Diagnosis:

```
ls -l file                       # Show permissions and owner
ls -ld directory/                # Show directory permissions
whoami                           # Show current user
groups                           # Show groups you belong to
```

sudo patterns:

```
sudo command                     # Run single command as root
sudo -i                          # Start root shell (exit to return)
sudo -u alice command            # Run command as alice
sudo !!                          # Re-run last command with sudo
```

Permission troubleshooting checklist:

1. Run `ls -l file` — check owner and permissions
2. Run `whoami` — check current user
3. **Script won't run?** → `chmod +x script`
4. **Can't read file?** → Check if you need read permission or ownership
5. **403 on web files?** → `sudo chown www-data:www-data file`
6. **SSH key rejected?** → `chmod 600 ~/.ssh/id_rsa`

Security best practices:

- ✅ Secrets: always `600` (owner only)
- ✅ Configs: usually `644` (owner writes, all read)
- ✅ Scripts: usually `755` (owner writes, all execute)
- ✅ SSH keys: private `600`, public `644`
- ✅ Never use `777` in production (world-writable = security hole)

Mission Complete: Access Control ✅

Three emergencies -- three one-line fixes:

Script won't run

`chmod +x deploy.sh`

Web page shows 403

`sudo chown www-data:www-data file`

Credentials exposed

`chmod 600 secrets.env`

Your new commands: `chmod` (change permissions), `chown` (change ownership), `sudo` (run as admin)

> Next mission: you'll customize your terminal -- aliases, environment tweaks, and shortcuts that make the command line feel like home.

Questions?